AGENTIC ARCHITECT CHALLENGE

Architecture, Trade-offs and Failure Handling

A reliability-first design across customer support, web summarization, and document Q&A;

SYSTEM ARCHITECTURE

PART 1 SUPPORT EMAIL AGENT

Flow: Email -> normalize -> history -> critical guard -> classify -> retrieve evidence -> policy guard -> validate draft.

Agent decision: escalate before drafting for data loss, outage, breach, or more than 3 contacts in 7 days.

Grounding: category-aware PDF/FAQ retrieval; refund claims require the current approved refund policy.

PART 2 WEB SUMMARIZER

Flow: Validate URL -> static fetch -> main-content extraction -> Playwright fallback -> chunk -> map-reduce summary -> guardrail.

Reliability: SSRF checks, redirect validation, timeouts, response limits, and controlled errors.

Guardrail: maximum 150 words and 5 bullets, with compression and a deterministic final boundary.

PART 3 DOCUMENT AGENT

Flow: Load one sample FAQ -> Gemini conversation -> document answer or calculator call -> append result to session history.

Memory: retains the user's name and previous context during the terminal session.

Tool choice: calculator is exposed in auto mode and runs only when arithmetic is needed.

Evidence before generation | Bounded retries and limits | Human-safe failure paths | Structured logs and tests

KEY TRADE-OFFS

Rules vs semantics: deterministic checks are auditable; semantic detection improves recall but can misclassify.
Speed vs coverage: static HTTP is fast; Playwright handles JavaScript pages at higher CPU and latency cost.
Context vs cost: overlapping chunks preserve meaning but increase processing work.
Automation vs safety: stricter evidence rules increase human escalation but reduce unsupported policy claims.

FAILURE POINTS AND CONTROLS

Missing or uncertain evidence: stop generation and route to a human.
Fetch, browser, or model failure: bounded timeout/retry, then controlled error.
Unsafe URL or oversized content: reject before processing.
Invalid tool output or repeated calls: safe parser and three-call ceiling.
Operational drift: version prompts/knowledge, log decisions, measure latency, errors, and overrides.

VERIFIED 18 automated tests passed | Static and Playwright scraping verified | Gemini memory, document grounding, and conditional calculator use verified

Implementation: Python, HTTPX, Trafilatura, Playwright, Gemini API compatibility, pytest

Secrets are loaded from .env; .env is excluded from version control.